AS 1 · AP01

System

AS 1 · AP01

Sub-system

AS 1 · AP01

User interface (UI)

AS 1 · AP01

Data

AS 1 · AP01

Process

AS 1 · AP01

Software crisis

AS 1 · AP01

Software engineering

AS 1 · AP01

Systems development life cycle (SDLC)

Backs · print double-sided, flip on long edge

A smaller system that forms part of a larger system and performs one part of its overall function, often exchanging data with other sub-systems.

Sub-system

A set of interdependent components that work together for a stated purpose, satisfying an identified need.

System

Raw facts and figures that have no context or meaning on their own, which are input to or stored by a system.

Data

The means by which a user interacts with a computer system, entering data into it and viewing the results of processing.

User interface (UI)

The period from the mid-1960s when software projects were frequently late, over budget, unreliable and failed to meet user requirements because development methods could not keep pace with demand.

Software crisis

An operation carried out by a computer system that manipulates data, for example calculating, sorting, searching or updating, converting data into information.

Process

A structured framework of stages (feasibility, analysis, design, development, testing, implementation, maintenance and evaluation) used to plan, build and maintain an information system.

Systems development life cycle (SDLC)

Using a disciplined, systematic engineering process to design, build and maintain software.

Software engineering

AS 1 · AP01

Legacy system

AS 1 · AP01

Deliverable

AS 1 · AP01

Information system

AS 1 · AP01

Analysis

AS 1 · AP02

Feasibility study

AS 1 · AP02

Feasibility report

AS 1 · AP02

Operational feasibility

AS 1 · AP02

Cost–benefit analysis

Backs · print double-sided, flip on long edge

A document or product, such as a feasibility report or test plan, that is produced and signed off at the end of a stage of a project.

Deliverable

An old system that is still in use but is often unreliable, unsupported or expensive to maintain.

Legacy system

The stage in which the current system is investigated using fact-finding methods to establish the problems and the requirements of the new system.

Analysis

A system of people, procedures, hardware, software and data that collects, processes, stores and distributes information to support an organisation.

Information system

The document produced by a feasibility study that describes the options considered, their feasibility and a justified recommendation for management.

Feasibility report

A preliminary investigation to decide whether a proposed system is technically, economically, legally, operationally and schedule feasible, ending in a recommendation.

Feasibility study

A comparison of the estimated costs of a proposed system with its expected tangible and intangible benefits over a period of time.

Cost–benefit analysis

Whether a proposed system will work in practice in the organisation and be accepted and used by its staff and customers.

Operational feasibility

AS 1 · AP02

Payback period

AS 1 · AP02

Intangible benefit

AS 1 · AP02

Tangible benefit

AS 1 · AP02

User needs

AS 1 · AP02

Systems analyst

AS 1 · AP02

Project manager

AS 1 · AP02

Programmer

AS 1 · AP03

Analysis

Backs · print double-sided, flip on long edge

A benefit that is difficult to measure in money, such as improved customer satisfaction or staff morale.

Intangible benefit

The length of time it takes for the cumulative benefits of a system to equal its cumulative costs.

Payback period

The requirements that an organisation and the individuals who use a system have of it: what it must do and how they need to interact with it.

User needs

A benefit that can be measured in money, such as reduced staffing or postage costs.

Tangible benefit

The person responsible for planning, co-ordinating and controlling a project so that it is delivered on time, within budget and meets its objectives.

Project manager

The person who identifies the user and data processing requirements of a new or modified system, carrying out the feasibility study, fact-finding and producing the system specification.

Systems analyst

The stage of systems development in which the existing system is investigated and the requirements for its replacement are identified and agreed with the client.

Analysis

The person who writes, documents, tests and debugs the source code for the system designed by the systems analyst.

Programmer

AS 1 · AP03

Fact-finding

AS 1 · AP03

Systems analyst

AS 1 · AP03

Direct data source

AS 1 · AP03

Indirect data source

AS 1 · AP03

Hawthorne effect

AS 1 · AP03

Closed question

AS 1 · AP03

Open question

AS 1 · AP03

Document sampling

Backs · print double-sided, flip on long edge

The person who investigates the current system, establishes the requirements and specifies the new system.

Systems analyst

Gathering information about a current system, using techniques such as interviews, questionnaires, observation and document sampling, to establish how it works and what is required.

Fact-finding

Data that already exists and is reused for a purpose other than the one for which it was originally collected.

Indirect data source

A source of data collected first-hand specifically for the current purpose, such as a survey designed for the project.

Direct data source

A question with a fixed set of possible answers, producing quantitative data that is easy to analyse.

Closed question

The tendency of people to change their behaviour because they know they are being observed, which can make observation unreliable.

Hawthorne effect

Examining a sample of existing forms, invoices, reports and records to identify the data items, formats, volumes and outputs of the current system.

Document sampling

A question that lets the respondent answer in their own words, giving richer qualitative data that is harder to analyse.

Open question

AS 1 · AP03

Triangulation

AS 1 · AP04

Stakeholder

AS 1 · AP04

End user

AS 1 · AP04

Functional requirement

AS 1 · AP04

Non-functional requirement

AS 1 · AP04

Quality attribute

AS 1 · AP04

System specification

AS 1 · AP04

MoSCoW

Backs · print double-sided, flip on long edge

A person or group with an interest in a system or who is affected by it, such as managers, end users, customers and regulators.

Stakeholder

Checking a finding from one source of evidence against another to confirm that it is accurate and consistent.

Triangulation

A statement of something the system must do, such as an input, process, calculation or output.

Functional requirement

A person who operates the system directly in their day-to-day work or as a customer.

End user

A measurable characteristic used to judge a system, such as performance, reliability or security; non-functional requirements specify these.

Quality attribute

A condition or constraint on how the system behaves, such as response time, availability, reliability, security or usability, rather than a function it performs.

Non-functional requirement

A method of prioritising requirements as Must have, Should have, Could have and Won't have this time.

MoSCoW

The document produced at the end of analysis that states precisely what the client expects the system to do, including requirements, constraints and schedule, and is agreed by the client.

System specification

AS 1 · AP04

Constraint

AS 1 · AP04

Testable requirement

AS 1 · AP05

Data flow diagram (DFD)

AS 1 · AP05

Context diagram

AS 1 · AP05

Level 1 DFD

AS 1 · AP05

External entity

AS 1 · AP05

Source

AS 1 · AP05

Sink

Backs · print double-sided, flip on long edge

A requirement stated precisely enough, usually with a measure, that it can be shown to have been met or not.

Testable requirement

A limit within which the system must be developed or operate, such as budget, deadline, hardware or legislation.

Constraint

A top-level (Level 0) DFD showing the whole system as a single process with its external entities and the data flows between them, defining the system boundary.

Context diagram

A diagram that models how data moves through a system using external entities, processes, data stores and labelled data flows.

Data flow diagram (DFD)

A person, organisation or system outside the system boundary that is a source or destination of data.

External entity

A DFD in which the single process of the context diagram is exploded into its main numbered sub-processes, with data stores and internal flows added.

Level 1 DFD

An external entity to which the system sends data.

Sink

An external entity from which data enters the system.

Source

AS 1 · AP05

Process

AS 1 · AP05

Data store

AS 1 · AP05

Data flow

AS 1 · AP05

Balancing

AS 1 · AP05

Explode

AS 1 · AP06

Design document

AS 1 · AP06

Storyboarding

AS 1 · AP06

Prototyping

Backs · print double-sided, flip on long edge

A place in a DFD where data is held for later use, such as a file or database table. Data store	An activity in a DFD that transforms input data into output data; named with a verb phrase and numbered. Process
The rule that the data flows into and out of a process (or the whole system) must be the same on its exploded lower-level DFD. Balancing	A labelled arrow in a DFD showing data moving between entities, processes and data stores. Data flow
The output of the design phase, setting out the user interface, inputs, outputs and reports, data structures, validation and verification, algorithms and an initial test plan for the new system. Design document	To break a DFD process down into more detailed sub-processes on a lower-level diagram. Explode
Building an early, incomplete working version of a system so that users can experience its look and feel and give feedback on the interface and data requirements. Prototyping	A design technique in which each screen of a proposed interface is sketched, showing its contents, inputs and outputs and the navigation between screens, to obtain early feedback from users. Storyboarding

AS 1 · AP06

Stub

AS 1 · AP06

Data dictionary

AS 1 · AP06

Validation

AS 1 · AP06

Verification

AS 1 · AP06

Human–computer interaction (HCI)

AS 1 · AP06

Consistency check

AS 1 · AP06

Design specification

AS 1 · AP07

Unit testing

Backs · print double-sided, flip on long edge

A central record of every data item in a system, giving its name, data type, length, format, description, example and validation rules.

Data dictionary

A part of a prototype or program, such as a menu option or subroutine, that is present but has no working functionality yet.

Stub

A check that data has been entered or transferred accurately, such as double entry or proof-reading against the source.

Verification

An automatic check by the system that data is reasonable and follows set rules before it is accepted; it cannot ensure the data is correct.

Validation

A validation check that compares two or more fields to make sure their values are compatible, e.g. return date after loan date.

Consistency check

The study and design of how people interact with computer systems so that interfaces are usable, efficient and accessible.

Human–computer interaction (HCI)

Testing an individual module or subroutine on its own, usually in-house by the programmer, to check it works correctly in all circumstances.

Unit testing

The deliverable of the design stage describing the inputs, outputs, interfaces, data structures, processes, security and test plan of the new system.

Design specification

AS 1 · AP07

Integration testing

AS 1 · AP07

System testing

AS 1 · AP07

Alpha testing

AS 1 · AP07

Beta testing

AS 1 · AP07

Acceptance testing

AS 1 · AP07

Black-box testing

AS 1 · AP07

White-box testing

AS 1 · AP07

Test plan

Backs · print double-sided, flip on long edge

Testing the complete, integrated system against its agreed specification; essentially black-box testing of the whole system.

System testing

Testing units that have already been tested once they are combined, to check that they interact correctly and pass data between them as expected.

Integration testing

Testing of a pre-release version by a group of potential external users in a realistic environment, without developers present, who report any faults.

Beta testing

The final phase of in-house testing carried out by the developer, following a test plan, before the software is released to users.

Alpha testing

Testing that checks outputs against expected results for given inputs without any access to or knowledge of the program code.

Black-box testing

The final test, by end users in their real working environment with genuine data at realistic volumes, to confirm the system meets their requirements before they sign it off.

Acceptance testing

A document listing every test to be carried out, giving the part of the system tested, what is tested, the test data and the expected outcome, with space for the actual outcome and pass/fail.

Test plan

Testing that uses knowledge of the program code to design tests for every path, branch and loop.

White-box testing

AS 1 · AP07

Normal data

AS 1 · AP07

Boundary data

AS 1 · AP07

Erroneous data

AS 1 · AP08

Implementation

AS 1 · AP08

Changeover

AS 1 · AP08

Parallel changeover

AS 1 · AP08

Direct changeover

AS 1 · AP08

Pilot changeover

Backs · print double-sided, flip on long edge

Valid test data at the very edges of an allowed range, used to find errors in range limits. Boundary data	Typical valid data, correct in type and format, that the system should accept and process. Normal data
The stage at which a new system is put into operation, including installation, data conversion, training, documentation and changeover from the old system. Implementation	Test data that is wrong in type, format or range and should be rejected by the system. Erroneous data
A changeover method in which the old and new systems run at the same time on the same transactions so that outputs can be compared until the new system is proven. Parallel changeover	The process of moving from an old system to a new system, using a planned method: parallel, direct, pilot or phased. Changeover
A changeover method in which the whole new system is introduced in one branch, department or site before being extended to every other part of the organisation. Pilot changeover	A changeover method in which the old system stops and the new one takes over at a single moment, with no overlap. Direct changeover

AS 1 · AP08

Phased changeover

AS 1 · AP08

Data conversion

AS 1 · AP08

Cascade training

AS 1 · AP09

User documentation

AS 1 · AP09

Technical documentation

AS 1 · AP09

System maintenance

AS 1 · AP09

Corrective maintenance

AS 1 · AP09

Adaptive maintenance

Backs · print double-sided, flip on long edge

Transferring existing records into the structure and format the new system expects, including keying in or scanning paper records and verifying and validating the result.

Data conversion

A changeover method in which the new system is introduced one component or function at a time.

Phased changeover

Non-technical documentation for end users, such as an overview, installation instructions, user guide, troubleshooting, FAQs and backup procedures.

User documentation

A training method in which a small number of staff are trained and then train their colleagues.

Cascade training

Any change applied to a system during its lifespan after it has been implemented.

System maintenance

Documentation for administrators and maintenance staff, including specifications, DFDs and data models, I/O and report designs, annotated code, test plans and results, and hardware/software configuration.

Technical documentation

Changes made so that the system can still carry out its tasks after a change in circumstances, internal or external, such as new legislation, a new tax rate or a new business policy.

Adaptive maintenance

Changes made to fix flaws, errors or weaknesses, including security flaws, discovered after the system is in use.

Corrective maintenance

AS 1 · AP09

Perfective maintenance

AS 1 · AP09

Software patch

AS 1 · AP09

Annotated code

AS 1 · AP09

Post-implementation review

AS 1 · AP10

Waterfall model

AS 1 · AP10

Deliverable

AS 1 · AP10

System specification

AS 1 · AP10

Design document

Backs · print double-sided, flip on long edge

A small update released to correct, change or enhance a program or its data, often downloaded and installed automatically. Software patch	Changes that improve the performance or usability of a system that already works, after initial delivery. Perfective maintenance
A formal evaluation, after a system has been in use for some time, of how well it meets its original objectives and requirements. Post-implementation review	Program code that includes comments explaining its purpose and logic, so that others can understand and maintain it. Annotated code
The end product of a stage of development, such as a system specification or design document, which is passed on as the input to the next stage. Deliverable	A sequential development methodology in which distinct stages are completed in order, each producing a deliverable and being signed off before the next stage begins. Waterfall model
The deliverable of the design stage, describing processes, inputs and outputs, software units, user interfaces and hardware precisely enough for programmers to code from. Design document	The deliverable of the requirements stage: a clear, unambiguous and complete document of the user requirements for the new system. System specification

AS 1 · AP10

Handover

AS 1 · AP10

Iterative development

AS 1 · AP10

Spiral model

AS 1 · AP10

Risk analysis

AS 1 · AP10

Sign-off

AS 1 · AP11

Prototype

AS 1 · AP11

Throwaway prototyping

AS 1 · AP11

Evolutionary prototyping

Backs · print double-sided, flip on long edge

An approach in which a system is built through repeated cycles of planning, design, coding and testing, each producing a version that users review.

Iterative development

The point at which work passes from one waterfall stage, and often one group of staff, to the next; misunderstandings introduced here may travel undetected through later stages.

Handover

Identifying possible threats to a project’s success, assessing their likelihood and impact, and deciding how to reduce or resolve them.

Risk analysis

A risk-driven iterative model in which each loop determines objectives, analyses and resolves risks, develops and tests the next version, and plans the next loop.

Spiral model

An early model of all or part of a system, built quickly so that users can evaluate it and clarify their requirements.

Prototype

Formal agreement by the client or manager that a stage’s deliverable is complete and accepted, allowing the next stage to begin.

Sign-off

An approach in which a prototype is repeatedly refined in response to user feedback until it becomes the final system.

Evolutionary prototyping

An approach in which a prototype is used to establish requirements and is then discarded before the real system is built.

Throwaway prototyping

AS 1 · AP11

Rapid application development (RAD)

AS 1 · AP11

Timeboxing

AS 1 · AP11

Product backlog

AS 1 · AP11

Release backlog

AS 1 · AP11

Sprint

AS 1 · AP11

Scrum master

AS 1 · AP11

Daily Scrum

AS 1 · AP11

Burn down chart

Backs · print double-sided, flip on long edge

Fixing the time allowed for a project or part of it so that the deadline cannot be overrun; lower-priority features are dropped instead. Timeboxing	A methodology that delivers systems quickly through four phases (requirements, user design, construction, cutover), frequent prototypes, close user involvement and a strict timescale with prioritised requirements. Rapid application development (RAD)
The features chosen from the product backlog for the next release, broken down into work items. Release backlog	In Agile, the prioritised wish list of all the features the product should eventually provide. Product backlog
The team member who ensures the Scrum team works effectively, arranges meetings, monitors progress and removes obstacles. Scrum master	In Agile, a fixed period of a few weeks in which the team produces a working increment of the product. Sprint
A graph of the work remaining in a sprint against time, compared with an ideal straight line, used to check whether work is on schedule. Burn down chart	A short daily meeting of about 15 minutes in which each team member says what they did yesterday, what they will do today and what obstacles they face. Daily Scrum

AS 1 · AP11

Pair programming

AS 1 · AP12

Resources

AS 1 · AP12

Elastic resource

AS 1 · AP12

Constraint

AS 1 · AP12

Risk

AS 1 · AP12

Requirements shift

AS 1 · AP12

Project manager

AS 1 · AP12

SMART objectives

Backs · print double-sided, flip on long edge

Anything a project can use to meet its objectives: human (staff), technical (e.g. processing power), financial (budget) or physical (e.g. accommodation).

Resources

An extreme programming practice in which two programmers work at one computer, one writing code and the other reviewing it, swapping roles regularly.

Pair programming

A restriction on a project that the developer cannot change, usually concerning scope, time or resources.

Constraint

A resource whose amount can be increased or decreased, such as the budget or number of staff; an inelastic resource, such as time before a hard deadline, cannot.

Elastic resource

Changes to what the client wants after the project has started, often caused by inadequate user involvement during analysis.

Requirements shift

Anything that could have a harmful effect on a project or cause it to fail, such as missed deadlines, requirements shift, staff turnover or low productivity.

Risk

Objectives that are Specific, Measurable, Agreed, Realistic and Timed, so that all parties share the same view of success.

SMART objectives

The person responsible for planning, scheduling, resourcing, monitoring and managing risk so that a project is delivered on time, within budget and to the required quality.

Project manager

AS 1 · AP12

Gantt chart

AS 1 · AP12

Critical path

AS 1 · AP12

Earliest start time (EST)

AS 1 · AP12

Latest finish time (LFT)

AS 1 · AP12

Float (slack)

AS 1 · AP13

Version management

AS 1 · AP13

Repository

AS 1 · AP13

Commit

Backs · print double-sided, flip on long edge

The longest sequence of dependent tasks in a project, which determines the minimum time in which the project can be completed.

Critical path

A horizontal bar chart showing project tasks against a timeline, including their start and end, duration, overlaps, dependencies and milestones.

Gantt chart

In critical path analysis, the latest time by which an activity must be complete if the project is not to be delayed.

Latest finish time (LFT)

In critical path analysis, the earliest time a task can start, once every task it depends on has finished.

Earliest start time (EST)

The process by which a development team keeps track of changes to code files over time, recording who made each change, when and why, and allowing return to earlier versions.

Version management

The amount of time a non-critical task can be delayed without delaying the completion of the whole project.

Float (slack)

Saving a change to the repository as a new version, together with a message describing the change.

Commit

The central store in which a version management system holds all of a project's files and their complete history of changes.

Repository

AS 1 · AP13

Branch

AS 1 · AP13

Merge

AS 1 · AP13

Rollback

AS 1 · AP13

Version management graph

AS 1 · AP13

CASE tool

AS 1 · AP13

Code review

AS 1 · AP13

Quality assurance

AS 1 · AP13

Coding standards

Backs · print double-sided, flip on long edge

Combining two parallel versions or branches into a single version that contains both sets of changes.

Merge

In version management, a separate line of development used to work on a feature or fix without affecting the main version.

Branch

A diagram of a project's history in which nodes are versions and arrows show which version each was created from, showing branches, merges and rollbacks.

Version management graph

Returning to an earlier, stable version of the code and continuing development from that point.

Rollback

The examination of code by other developers to find errors, security weaknesses and breaches of coding standards before it is accepted.

Code review

Computer-aided software engineering tool: software that supports or automates parts of development, such as diagramming (upper CASE) or code generation and debugging (lower CASE).

CASE tool

Agreed rules for writing code, such as naming conventions, layout, commenting and error handling, that make code consistent and maintainable.

Coding standards

Planned and systematic activities, such as reviews, inspections, testing and audits, that build quality into every stage of a project.

Quality assurance

AS 1 · AP14

Malware

AS 1 · AP14

Worm

AS 1 · AP14

Trojan

AS 1 · AP14

Ransomware

AS 1 · AP14

Phishing

AS 1 · AP14

Social engineering

AS 1 · AP14

Distributed denial of service (DDoS)

AS 1 · AP14

Insider threat

Backs · print double-sided, flip on long edge

Standalone malware that replicates itself across a network without needing a host file or any user action, usually by exploiting unpatched vulnerabilities. Worm	Malicious software, such as viruses, worms, Trojans, ransomware and spyware, written to damage or disrupt systems or to gain unauthorised access to data. Malware
Malware that encrypts a victim's files and demands payment in exchange for the decryption key. Ransomware	Malware disguised as legitimate software that the user installs; it carries a hidden harmful payload such as a back door and does not self-replicate. Trojan
Manipulating people into giving away confidential information or access, rather than attacking the technology directly. Social engineering	A social engineering attack using fraudulent emails, messages or websites that imitate trusted organisations to trick people into revealing personal details. Phishing
A risk to data posed by people inside an organisation, such as employees or contractors, who misuse their legitimate access deliberately or carelessly. Insider threat	An attack in which many compromised devices flood a server or network with requests so that legitimate users cannot access it. Distributed denial of service (DDoS)

AS 1 · AP14

Data archiving

AS 1 · AP14

Inactive data

AS 1 · AP14

Data recovery

AS 1 · AP14

Restore point

AS 1 · AP15

Security policy

AS 1 · AP15

Authentication

AS 1 · AP15

Multi-factor authentication (MFA)

AS 1 · AP15

Encryption

Backs · print double-sided, flip on long edge

Data that is no longer needed for day-to-day operations but must be kept for reference, audit or legal reasons. Inactive data	Moving inactive data that is no longer needed for daily use from the live system to separate, secure long-term storage, indexed so it can be retrieved later. Data archiving
A record of the state of a system at the precise moment it was backed up, allowing the system to be returned to that state. Restore point	Using backed-up data to restore a system to a previously correct state after data has been lost or corrupted. Data recovery
The process of proving that a user is who they claim to be, using something they know, have or are. Authentication	A formal document setting out the rules, procedures and responsibilities an organisation uses to protect its systems and data. Security policy
Converting plaintext into ciphertext using an algorithm and a key so that it cannot be understood without the correct key. Encryption	Authentication requiring two or more different types of evidence, such as a password and a code sent to a phone. Multi-factor authentication (MFA)

AS 1 · AP15

Firewall

AS 1 · AP15

Full backup

AS 1 · AP15

Differential backup

AS 1 · AP15

Incremental backup

AS 1 · AP15

Disaster recovery plan

AS 1 · AP15

Recovery point objective (RPO)

AS 1 · AP15

Recovery time objective (RTO)

AS 1 · AP15

Downtime

Backs · print double-sided, flip on long edge

A backup that copies every file on the system, whether or not it has changed, each time it runs.

Full backup

Hardware or software that monitors network traffic and blocks data that does not meet a set of security rules.

Firewall

A backup that copies only files modified after the most recent backup of either kind, full or incremental.

Incremental backup

A backup that copies every file altered after the most recent full backup, so it grows until the next full backup.

Differential backup

In the CCEA textbook, the list of parts of the system that must be restored after a disaster; more widely, the maximum amount of data, measured in time, that can acceptably be lost.

Recovery point objective (RPO)

A documented plan setting out how an organisation will restore its IT systems and data and resume operating after a catastrophic incident such as fire, flood or a cyber attack.

Disaster recovery plan

The period during which a system is not operational and cannot be used.

Downtime

The time limit within which each part of the system must be recovered after a disaster.

Recovery time objective (RTO)

AS 1 · AP16

Computer program

AS 1 · AP16

Application program

AS 1 · AP16

Algorithm

AS 1 · AP16

Finiteness

AS 1 · AP16

Decomposition

AS 1 · AP16

Abstraction

AS 1 · AP16

Pseudocode

AS 1 · AP16

Flowchart

Backs · print double-sided, flip on long edge

Software designed to perform a specific task for the user, such as a word processor or spreadsheet.

Application program

A set of instructions that a computer executes in order to perform a specific task.

Computer program

The property that an algorithm must terminate after a limited number of steps.

Finiteness

A finite, ordered sequence of precise, unambiguous steps designed to solve a problem or complete a task.

Algorithm

Removing unnecessary detail from a problem so that only the relevant details remain.

Abstraction

Breaking a complex problem into smaller sub-problems that can be solved independently.

Decomposition

A diagram that represents an algorithm using standard symbols joined by flow lines showing the order of steps.

Flowchart

A structured, language-independent, English-like way of describing an algorithm that resembles program code.

Pseudocode

AS 1 · AP16

Trace table

AS 1 · AP16

Dry run

AS 1 · AP16

Sentinel value

AS 1 · AP16

Logic error

AS 1 · AP17

Linear search

AS 1 · AP17

Binary search

AS 1 · AP17

Divide and conquer

AS 1 · AP17

Time complexity

Backs · print double-sided, flip on long edge

Working through an algorithm or program by hand, step by step, to check its logic. Dry run	A table used during a dry run to record the value of each variable and any output at each step of an algorithm. Trace table
An error in the design of an algorithm that causes incorrect output although the program still runs. Logic error	A special input value, such as -1, that signals the end of the data and stops a condition-controlled loop. Sentinel value
A search of a sorted list that repeatedly compares the target with the middle item and discards the half that cannot contain it. Binary search	A search that checks each item in turn from the start until the target is found or the end of the list is reached. Linear search
A measure of how the number of steps an algorithm takes grows as the size of its input grows. Time complexity	An approach that solves a problem by repeatedly reducing it to smaller sub-problems, as when binary search halves the list each time. Divide and conquer

AS 1 · AP17

Big-O notation

AS 1 · AP17

Worst case

AS 1 · AP17

Best case

AS 1 · AP17

Integer division

AS 1 · AP17

Logarithmic time

AS 1 · AP18

Bubble sort

AS 1 · AP18

Insertion sort

AS 1 · AP18

Merge sort

Backs · print double-sided, flip on long edge

The input that makes an algorithm do the most work, such as the target being last or absent in a linear search.

Worst case

Notation that describes the order of growth of an algorithm's time or space, usually in the worst case, as input size increases, e.g. O(n).

Big-O notation

Division that discards any remainder, so 13 DIV 2 = 6; used to calculate the midpoint in a binary search.

Integer division

The input that makes an algorithm do the least work, such as the target being the first item checked.

Best case

A sort that repeatedly compares adjacent items and swaps them if they are in the wrong order until a pass makes no swaps.

Bubble sort

O(log n) growth, in which doubling the input size adds only a constant amount of extra work.

Logarithmic time

A divide-and-conquer sort that splits a list into single items and then merges sub-lists back together in order.

Merge sort

A sort that takes each item in turn and inserts it into its correct position within an already sorted section of the list.

Insertion sort

AS 1 · AP18

Quick sort

AS 1 · AP18

Divide and conquer

AS 1 · AP18

Pass

AS 1 · AP18

Pivot

AS 1 · AP18

Swapped flag

AS 1 · AP18

Temporary variable

AS 1 · AP18

In place

AS 1 · AP19

Data type

Backs · print double-sided, flip on long edge

An approach that solves a problem by repeatedly breaking it into smaller sub-problems of the same type and combining their solutions.

Divide and conquer

A divide-and-conquer sort that partitions items around a pivot and then recursively sorts each partition.

Quick sort

The item chosen in quick sort around which the other items are partitioned into smaller and larger groups.

Pivot

One complete traversal through the list (or the unsorted part of it) by a sorting algorithm.

Pass

A variable used to hold one value during a swap so that it is not overwritten and lost.

Temporary variable

A Boolean variable in a bubble sort that records whether any swap was made during a pass, so the sort can stop early.

Swapped flag

A classification of data that determines the values a variable can hold, the memory it needs and the operations that can be performed on it.

Data type

Describes a sort that rearranges items within the original array, needing little or no extra memory.

In place

AS 1 · AP19

Variable

AS 1 · AP19

Constant

AS 1 · AP19

Identifier

AS 1 · AP19

Syntax

AS 1 · AP19

Integrated development environment (IDE)

AS 1 · AP19

Assembler

AS 1 · AP19

Compiler

AS 1 · AP19

Interpreter

Backs · print double-sided, flip on long edge

A named value that is set when the program is written and cannot change while the program runs. Constant	A named location in memory that stores a single value which can change while the program runs. Variable
The set of rules that define how valid statements must be written in a programming language. Syntax	The name given to a variable, constant or subroutine so that it can be referred to in a program. Identifier
A translator that converts assembly language into machine code, each mnemonic usually becoming one machine code instruction. Assembler	A software application that provides the tools needed to develop programs, such as an editor, translator and debugging tools, in one place. Integrated development environment (IDE)
A translator that translates and executes a high-level program one statement at a time, producing no object code. Interpreter	A translator that converts the whole of a high-level program into object code before it is executed, reporting any errors found. Compiler

AS 1 · AP19

Modulus

AS 1 · AP19

Integer division

AS 1 · AP19

Casting

AS 1 · AP20

Sequence

AS 1 · AP20

Selection

AS 1 · AP20

Iteration

AS 1 · AP20

Count-controlled loop

AS 1 · AP20

Condition-controlled loop

Backs · print double-sided, flip on long edge

Division that gives only the whole-number part of the result, discarding any remainder (DIV).

Integer division

An operator (% in C#, MOD in pseudocode) that gives the remainder after integer division.

Modulus

Executing instructions one after another in the order in which they are written.

Sequence

Converting a value from one data type to another, e.g. (double)total or (int)7.9.

Casting

Repeating a set of instructions, either a set number of times or while/until a condition is met.

Iteration

A construct in which a condition determines which of two or more paths through the program is followed.

Selection

A loop that repeats while or until a condition is met, so the number of repetitions need not be known in advance.

Condition-controlled loop

A loop that repeats a fixed number of times known before it starts, e.g. a for loop.

Count-controlled loop

AS 1 · AP20

Nested loop

AS 1 · AP20

Nested selection

AS 1 · AP20

Sentinel value

AS 1 · AP20

Pre-condition loop

AS 1 · AP20

Post-condition loop

AS 1 · AP20

Infinite loop

AS 1 · AP21

Subroutine

AS 1 · AP21

Procedure

Backs · print double-sided, flip on long edge

An if statement placed inside another if or else block, so the inner decision is only made when the outer condition has a particular result.

Nested selection

A loop placed inside the body of another loop; the inner loop completes fully for each pass of the outer loop.

Nested loop

A loop that tests its condition before each pass, so the body may run zero times (while).

Pre-condition loop

A dummy value entered to mark the end of the data, which is not processed as part of the data.

Sentinel value

A loop whose terminating condition is never met, usually because the control variable is not changed in the body.

Infinite loop

A loop that tests its condition after each pass, so the body always runs at least once (do... while / REPEAT...UNTIL).

Post-condition loop

A subroutine that performs a task but does not return a value (void in C#).

Procedure

A named, self-contained block of code that performs a specific task and can be called from elsewhere in a program.

Subroutine

AS 1 · AP21

Function

AS 1 · AP21

Parameter

AS 1 · AP21

Argument

AS 1 · AP21

Passing by value

AS 1 · AP21

Passing by reference

AS 1 · AP21

Scope

AS 1 · AP21

Local variable

AS 1 · AP21

Global variable

Backs · print double-sided, flip on long edge

A variable in a subroutine's header that receives a value (argument) passed in when the subroutine is called.

Parameter

A subroutine that performs a task and returns a single value to the point where it was called.

Function

Passing a copy of an argument to a subroutine so that the original variable cannot be changed by it.

Passing by value

The actual value or variable supplied to a subroutine when it is called.

Argument

The region of a program in which a variable can be accessed.

Scope

Passing the address of (a reference to) a variable so that changes made in the subroutine affect the original variable.

Passing by reference

A variable that can be accessed from any part of the program and exists for the whole run.

Global variable

A variable declared inside a subroutine or block that can only be accessed there and exists only while it runs.

Local variable

AS 1 · AP21

Modular programming

AS 1 · AP21

Library routine

AS 1 · AP22

Data structure

AS 1 · AP22

Array

AS 1 · AP22

Index

AS 1 · AP22

Two-dimensional array

AS 1 · AP22

Static data structure

AS 1 · AP22

Dynamic data structure

Backs · print double-sided, flip on long edge

A prewritten, tested subroutine supplied with a language or obtained from elsewhere that can be called in a program.

Library routine

Dividing a program into separate, self-contained modules (subroutines), each performing one clearly defined task.

Modular programming

A static data structure that holds a fixed number of elements of the same data type under one identifier, each accessed by an index.

Array

A way of organising and storing a collection of related data items under one identifier so that they can be processed efficiently.

Data structure

An array whose elements are arranged in rows and columns and are accessed using two indexes, [row, column].

Two-dimensional array

The position number used to access an element of an array or list; in C# the first index is 0.

Index

A data structure whose size can grow or shrink while the program is running, such as a list.

Dynamic data structure

A data structure whose size is fixed when it is created and cannot change while the program runs, such as an array.

Static data structure

AS 1 · AP22

List

AS 1 · AP22

Record

AS 1 · AP22

Field

AS 1 · AP22

Text file

AS 1 · AP22

Delimiter

AS 1 · AP22

Persistent storage

AS 1 · AP23

Robust program

AS 1 · AP23

Validation

Backs · print double-sided, flip on long edge

A data structure that groups a fixed set of related fields, which may be of different data types, to describe one item.

Record

A dynamic data structure whose size can change at run time as items are added or removed, with methods such as Add and Remove.

List

A file of characters, usually organised in lines, stored on secondary storage so that data persists between program runs.

Text file

One named item of data within a record, such as a member's name or age.

Field

Storage that keeps data after the program ends or power is removed, such as a file on secondary storage.

Persistent storage

A character, such as a comma, used to separate the fields of a record stored on one line of a text file.

Delimiter

An automatic check that input data is reasonable and follows set rules before it is processed.

Validation

A program that copes with invalid input and unexpected events without crashing or producing incorrect results.

Robust program

AS 1 · AP23

Verification

AS 1 · AP23

Exception

AS 1 · AP23

Syntax error

AS 1 · AP23

Semantic error

AS 1 · AP23

Run-time error

AS 1 · AP23

Logic error

AS 1 · AP23

Extreme data

AS 1 · AP23

Breakpoint

Backs · print double-sided, flip on long edge

An object raised when a run-time error occurs, which stops the program unless it is handled in a try...catch structure.

Exception

A check that data has been entered or transferred accurately, for example by double entry or a visual check.

Verification

A statement that follows the syntax rules but has no valid meaning, such as assigning text to an integer variable.

Semantic error

An error that breaks the grammatical rules of the programming language, so the program cannot be compiled or run.

Syntax error

An error in the algorithm that causes the program to run but produce incorrect results.

Logic error

An error that occurs while the program is running when an impossible operation is attempted, such as division by zero, often causing a crash.

Run-time error

A marker set in the IDE that pauses execution at a chosen line so the program's state can be examined.

Breakpoint

Test data at the boundaries of the valid range, such as the highest and lowest values allowed.

Extreme data

AS 1 · AP23

Stepping

AS 1 · AP23

Call stack

AS 1 · AP24

Class

AS 1 · AP24

Object

AS 1 · AP24

Instantiation

AS 1 · AP24

Attribute

AS 1 · AP24

Method

AS 1 · AP24

Constructor

Backs · print double-sided, flip on long edge

A debugger display of the chain of subroutine calls that led to the current line of execution. Call stack	Executing a program one line at a time in the IDE, either entering called subroutines (step into) or running them as one step (step over). Stepping
A specific instance of a class, with its own values for the class's attributes. Object	A template or blueprint that defines the attributes and methods (behaviours) shared by all objects of one type. Class
A data item (field) declared in a class whose values describe the state of each object. Attribute	Creating a new object from a class, in C# using the new keyword, which calls the constructor. Instantiation
A special method with the same name as the class and no return type that runs when an object is created to initialise its attributes. Constructor	A subroutine defined in a class that describes an action an object can perform or that can be performed on it. Method

AS 1 · AP24

Encapsulation

AS 1 · AP24

Get method

AS 1 · AP24

Set method

AS 1 · AP24

Parameter list

AS 1 · AP24

Property (C#)

AS 1 · AP24

Class diagram

AS 1 · AP25

Inheritance

AS 1 · AP25

Base class

Backs · print double-sided, flip on long edge

A public method that returns the current value of a private attribute.

Get method

Combining data and the methods that act on it in a class, and hiding the data (private) so it can only be accessed through the class's methods.

Encapsulation

The part of a method header, in brackets, that states the data items (with types) the method must be given when it is called.

Parameter list

A public method that takes a new value as a parameter and uses it to change a private attribute, often after validating it.

Set method

A UML diagram showing a class as a box with its name, attributes and methods, using +, - and # for public, private and protected.

Class diagram

A class member with get and/or set blocks that gives controlled access to a private field.

Property (C#)

A general class whose attributes and methods are inherited by other classes.

Base class

The mechanism by which a new, more specialised class (subclass) takes on the attributes and methods of an existing, more general class (superclass) and can add or modify its own.

Inheritance

AS 1 · AP25

Derived class

AS 1 · AP25

Protected

AS 1 · AP25

Abstract class

AS 1 · AP25

Polymorphism

AS 1 · AP25

Overriding

AS 1 · AP25

Overloading

AS 1 · AP25

Event-driven programming

AS 1 · AP25

Event handler

Backs · print double-sided, flip on long edge

An access modifier that makes a member visible inside its own class and any subclasses but hidden from other code; shown as # in a class diagram.

Protected

A more specialised class that inherits from a base class.

Derived class

The ability of objects of different classes to respond to the same method call in different ways.

Polymorphism

A class that cannot be instantiated; objects must be created from its subclasses, which must implement any abstract methods it declares.

Abstract class

Defining several methods in the same class with the same name but different parameter lists.

Overloading

Redefining an inherited method in a subclass with the same name and parameters, so the subclass's version runs for its objects.

Overriding

A subroutine that is run automatically when a particular event occurs on a control.

Event handler

A programming paradigm in which the flow of the program is determined by events such as user actions, sensor outputs or messages.

Event-driven programming

AS 1 · AP25

Control

	An object on a graphical user interface, such as a button or text box, with properties, methods and events. Control